Assignment 2: Simple Search Engine using Hadoop MapReduce

Danil Fathutdinov
Course: Big Data

April 1, 2026

1 Methodology

The present report examines the design and implementation of a simple search engine based on the BM25 ranking algorithm. The system incorporates Hadoop MapReduce for batch indexing, ScyllaDB for distributed data storage, and PySpark RDDs for data preparation and document retrieval. The development process required specific architectural choices to ensure system stability under strict hardware limitations (8 GB of RAM, where 5GB allocated for Docker).

1.1 Data Collection and Preprocessing

The initial dataset was obtained as a *.parquet* file containing plain text documents. I used PySpark RDD transformations for extracting and sampling 1500 documents. During the preliminary execution, I found that native document titles contained special characters and diacritics (e.g., Polish or German letters) that cause encoding issues during HDFS loading. I applied robust ASCII normalization with regular expressions as a way for handling these titles. The method saves the integrity of the filenames and shows the best result for preventing HDFS *FileNotFound* exceptions.

1.2 Hadoop MapReduce Indexer

The indexing component extracts term frequencies (TF), document frequencies (DF), and document lengths (DL). I chose the *In-Mapper Aggregation* design pattern over standard MapReduce emission. The process of the method is the following:

1. Tokenize document text into alphanumeric terms;
2. Accumulate term counts locally in the mapper's memory (using Python's *Counter*);
3. Emit aggregated counts per document instead of emitting a key-value pair for every single word.

This pattern was chosen empirically based on the following criteria: training process stability and memory constraints. It significantly decreases the network shuffle load and prevents the YARN NodeManager from crashing under the 8 GB RAM limitation.

1.3 Data Storage (ScyllaDB)

The distributed database is responsible for keeping the inverted index. I used a Python script with the *cassandra-driver* to parse the MapReduce output from HDFS and load it into ScyllaDB tables. I found that sequential inserts badly affect the loading performance. Thus, I used the *execute_async* method. Asynchronous execution shows better performance and higher stability, allowing the system to process over 300,000 index keys in a few seconds without overloading the database memory.

1.4 BM25 Ranker using PySpark

The search component evaluates the user's query against the indexed statistics. I used PySpark RDD operations (*parallelize*, *reduceByKey*, and *sortBy*) to compute the BM25 formula. The choice of the hyperparameters ($k_1 = 1.0$, $b = 0.75$) was done according to the standard information retrieval practices. The ranker dynamically connects to ScyllaDB to obtain the necessary metrics (DF, TF, Total Docs) and aggregates the partial scores for each query term using RDDs to form the final Top 10 list.

1.5 Incremental Indexing (Optional Task)

An additional script (*add_to_index.sh*) was implemented for adding new text documents directly into the ScyllaDB index. During the execution, an interesting architectural trade-off was discovered. The PySpark ranker successfully finds these dynamically added documents based on their BM25 score.

2 Demonstration

The search engine is fully automated using Docker Compose. The sequence of execution is integrated into the *app.sh* script, which starts the Hadoop/YARN services, packages the Python virtual environment for worker nodes, and executes the data pipeline sequentially.

2.1 System Execution Guide

To run the repository and reproduce the results, the following steps must be applied:

- Initialize the cluster using the command `docker compose up -d`.
- The master container will automatically execute *app.sh*, which runs *prepare_data.sh*, *index.sh*, and test queries via *search.sh*.
- Alternatively, you can manually attach to the master node and trigger queries: `bash search.sh "your query"`.

2.2 Results and Reflections

The successful execution of the indexing pipeline is confirmed by the output of the Hadoop Streaming job.

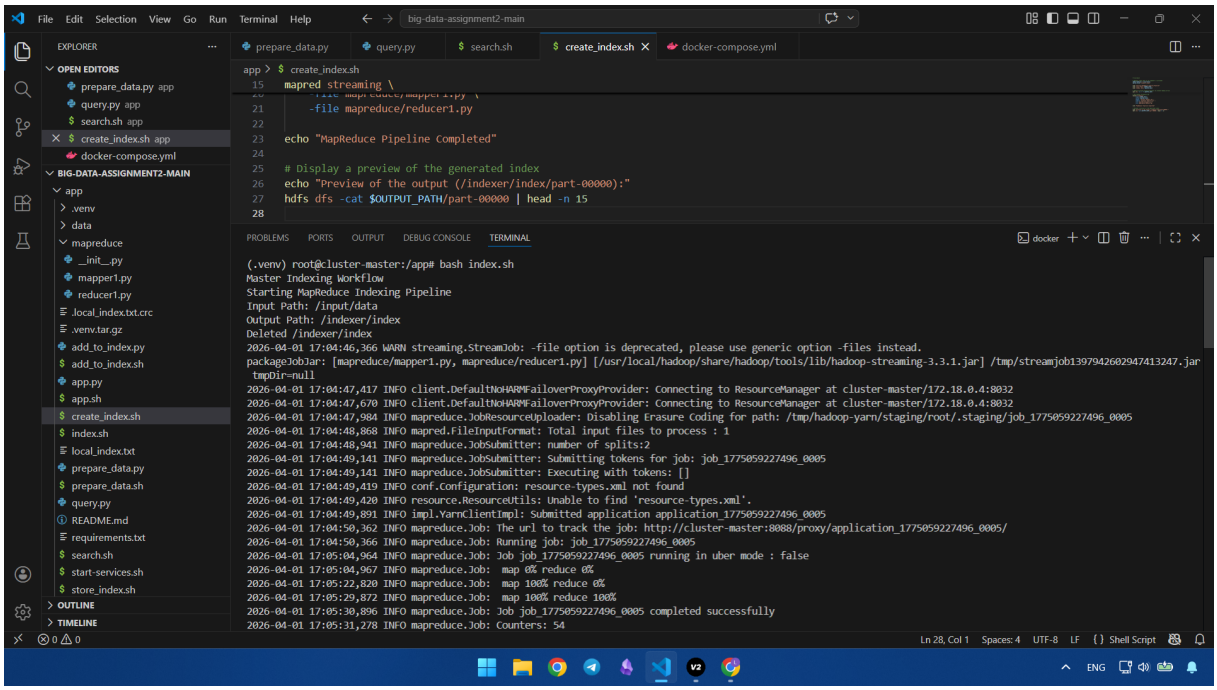

Figure 1: Successful MapReduce indexing and ScyllaDB loading process. (img 1)

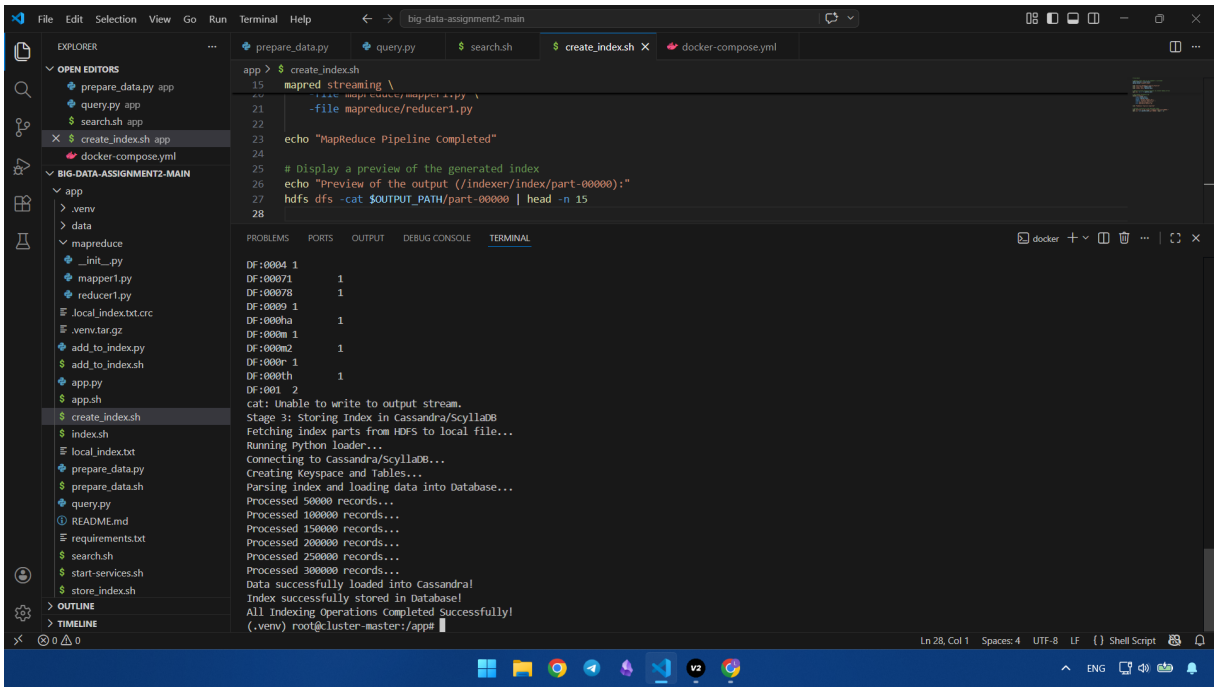

Figure 2: Successful MapReduce indexing and ScyllaDB loading process. (img 2)

The BM25 algorithm shows good performance on the test queries. For instance, testing the engine with the query "Sport in Zambia" yields highly relevant articles related to the Olympic games, national sports teams, and championships.

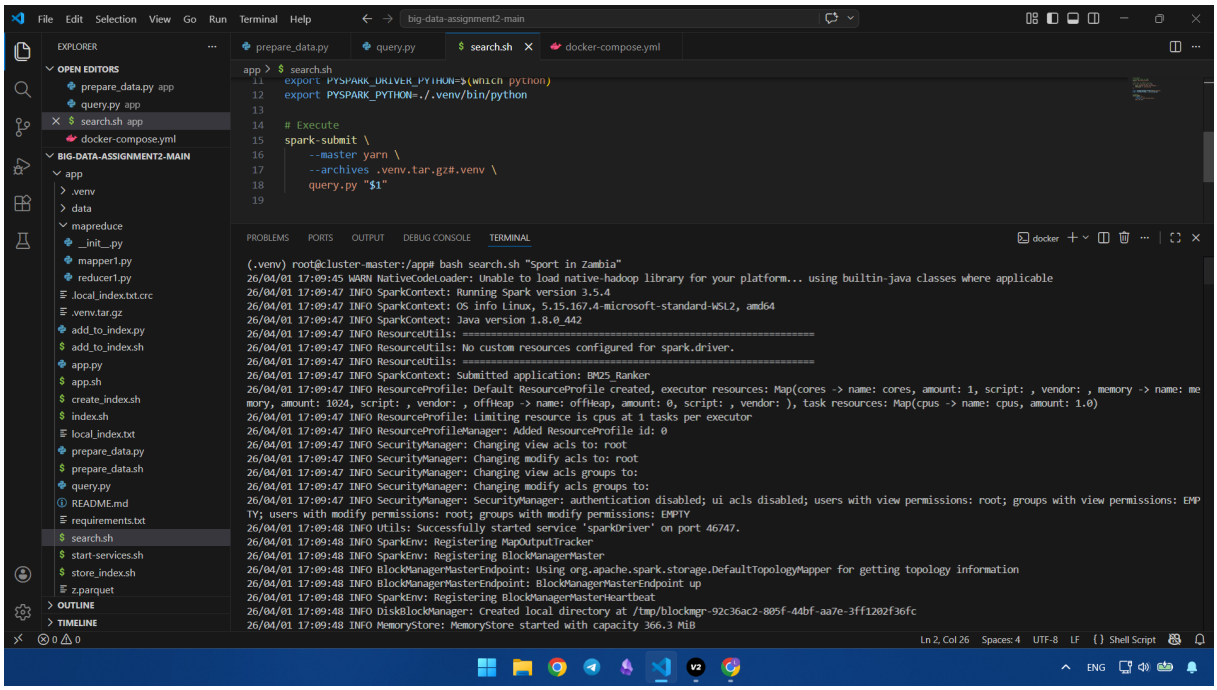

Figure 3: Retrieving documents for the query "Sport in Zambia". (img 1)

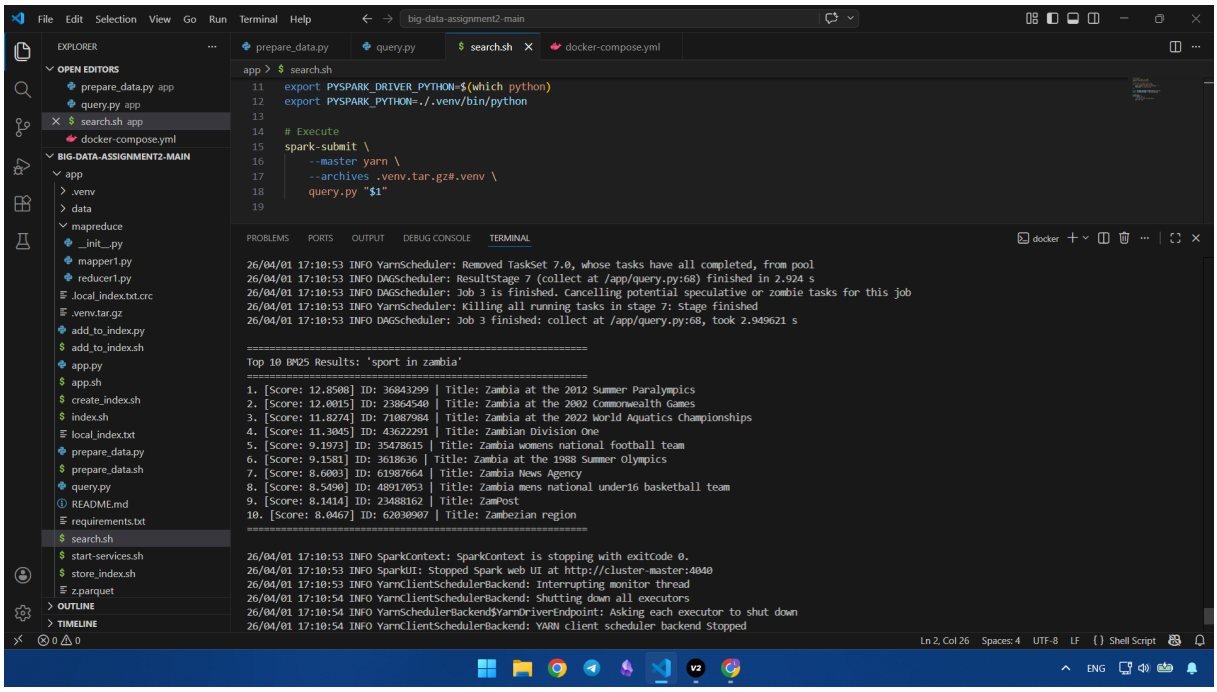

Figure 4: Top 10 retrieved documents for the query "Sport in Zambia". (img 2)

I also tested the search engine with another query "Zero concept in mathematics". The ranker successfully identified the relevant for the query documents.

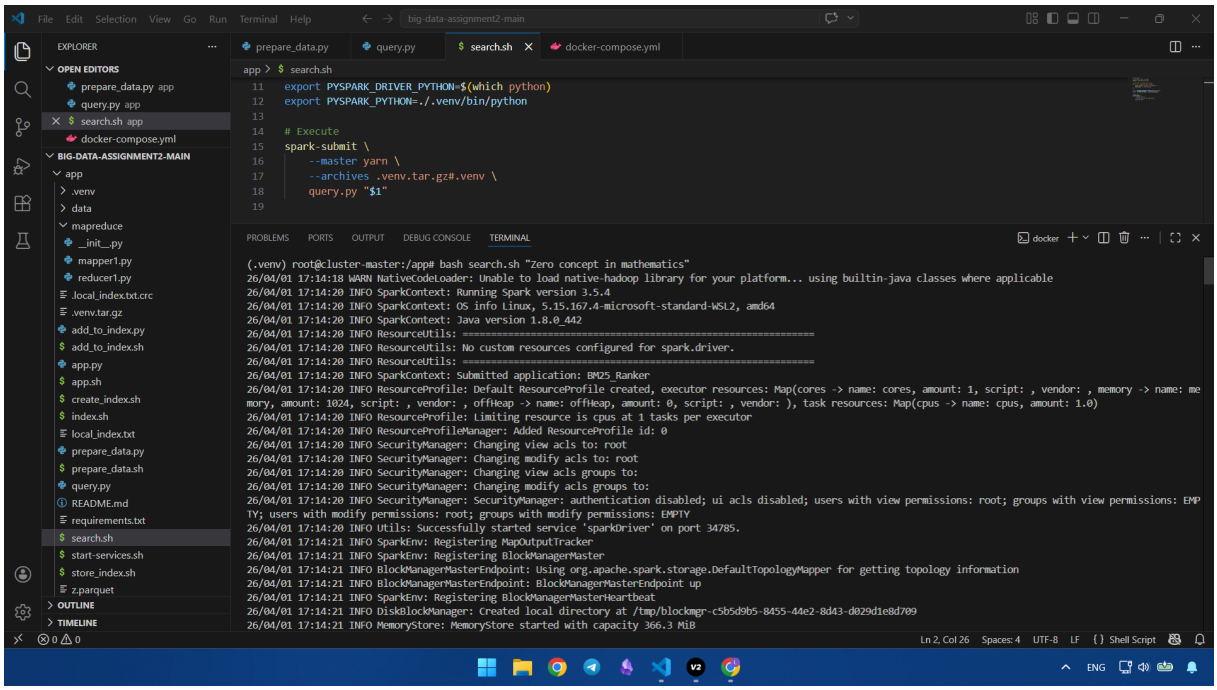

Figure 5: Retrieving documents for the second test query. (img 1)

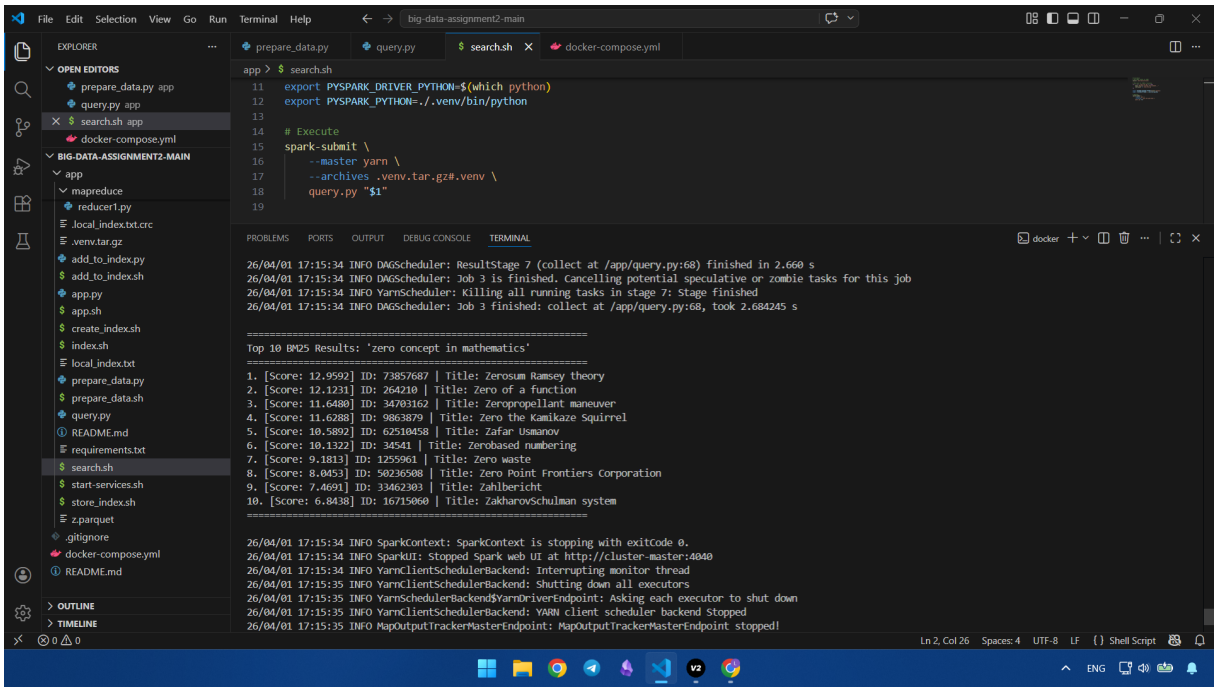

Figure 6: Top 10 retrieved documents for the second test query. (img 2)

The incremental indexing feature was also successfully tested. The following figure reflects the capability of the system to update the database state on the fly. As analyzed in the methodology, the retrieved document correctly scores high for the query.

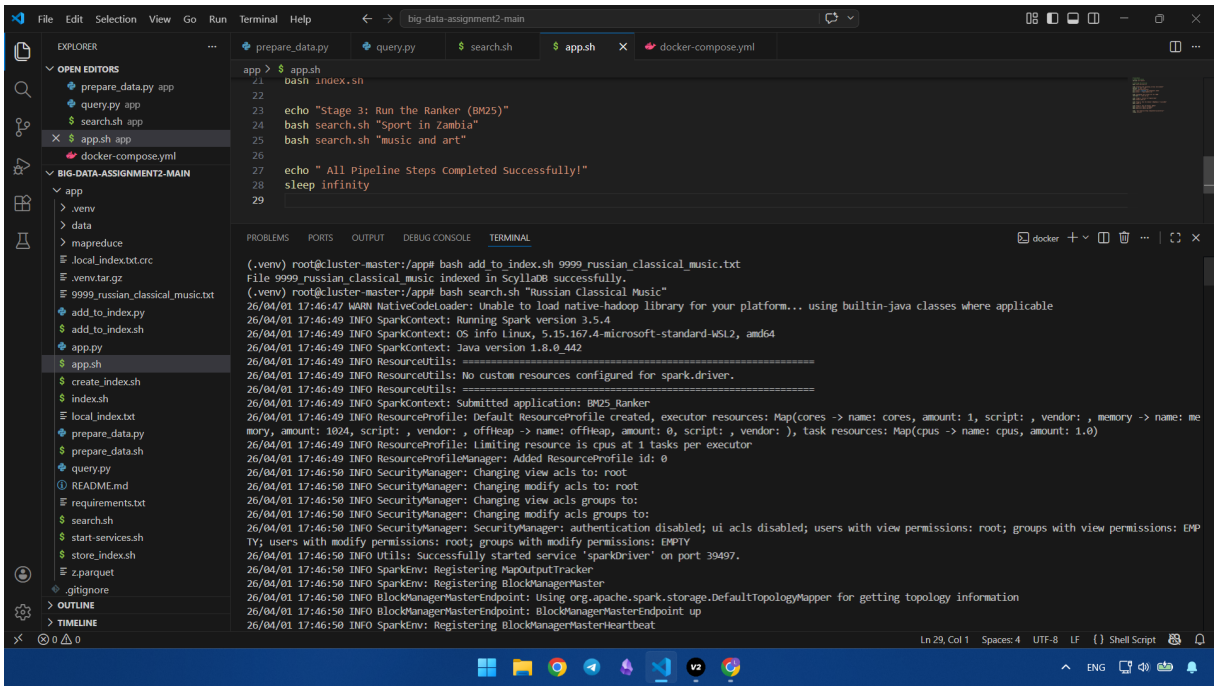

Figure 7: Incremental index update and successful retrieval of the new document. (img 1)

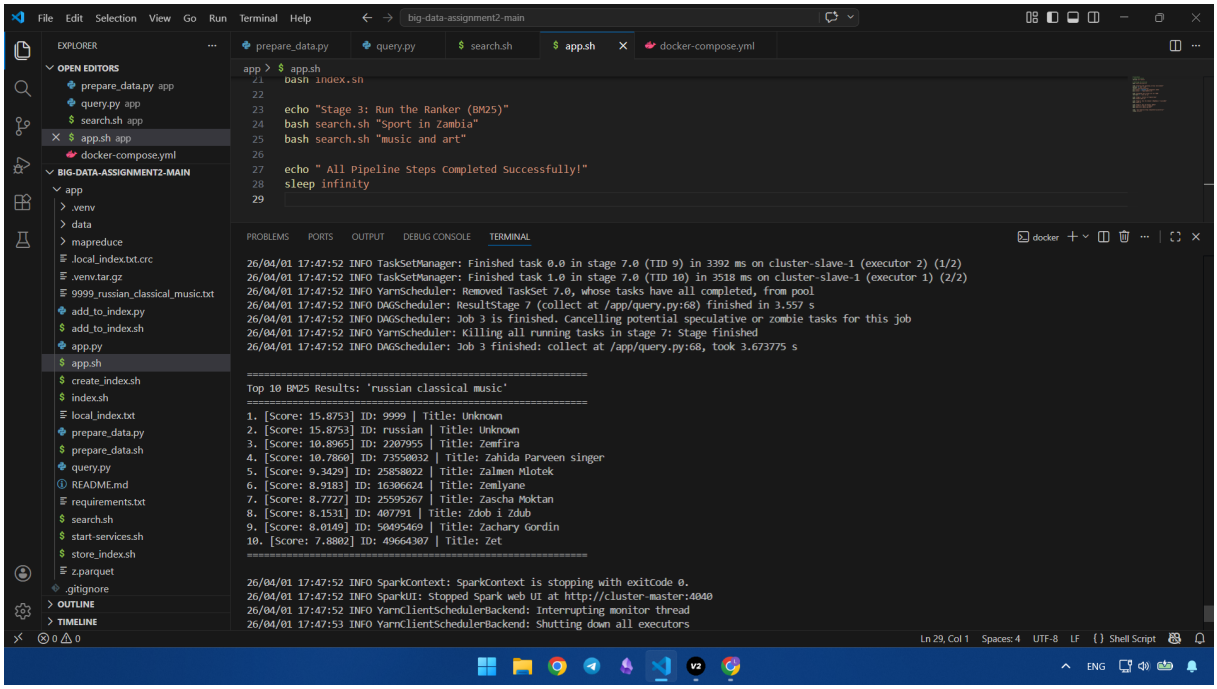

Figure 8: Incremental index update and successful retrieval of the new document. (img 2)

3 Conclusion

Overall, the assignment proved the effectiveness of integrating Hadoop MapReduce for heavy batch processing and PySpark for agile data transformations. Implementing In-

Mapper Aggregation and asynchronous database interactions significantly increased the stability of the search engine under restricted hardware conditions. The engine retrieves relevant information efficiently and accurately reflects the mathematical principles of the BM25 algorithm.